ASSIGNMENT 2

Text Analytics – Sentiment Analysis

Topic: Oppenheimer Movie Tweets

Student Name	Mulla Huzaifa Sufyan
Roll No	36
Subject	Text Analytics
Assignment	Assignment 2
Dataset Size	100 Tweets (80 Train / 20 Test)
Best Model	SVM (F1 = 0.4219)

1. Introduction

This report presents the findings of Assignment 2 on Text Analytics. The objective was to collect 100 tweets on the topic of the movie **Oppenheimer (2023)**, manually label them as **positive**, **neutral**, or **negative**, and apply three supervised machine learning classifiers to predict sentiment. The classifiers evaluated are: Naïve Bayes, Support Vector Machine (SVM), and Logistic Regression. Performance is measured using Precision, Recall, F1-Score, and Accuracy.

2. Dataset Description

100 tweets about Oppenheimer were manually collected and labelled. The dataset covers a wide range of opinions — fans praising the film's visuals and performances, mixed reactions about its pacing, and criticism about its length and moral stance. Labels were assigned based on the overall tone of each tweet.

Label	Count	Percentage	Train	Test
Positive	40	40%	32	8
Neutral	30	30%	24	6
Negative	30	30%	24	6
Total	**100**	**100%**	**80**	**20**

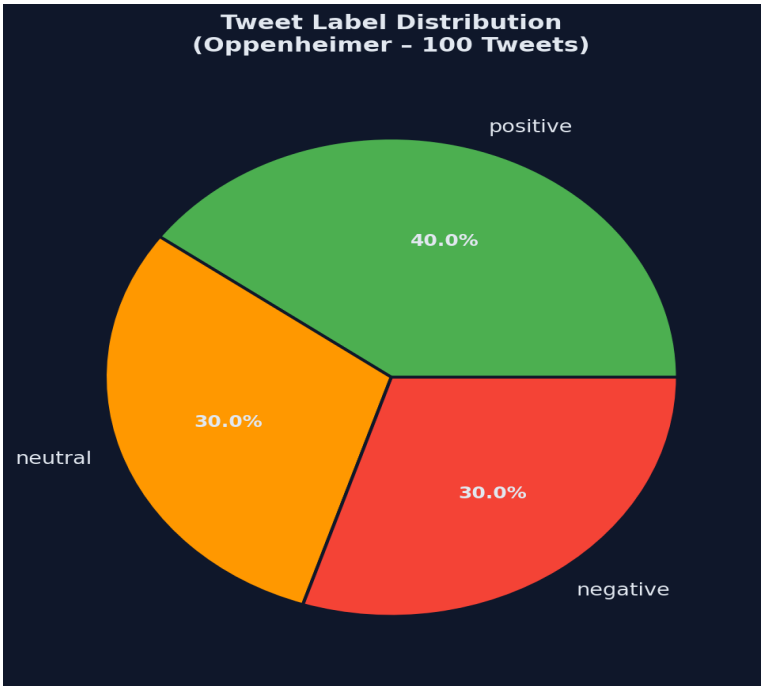

Figure 1: Tweet Label Distribution

3. Text Preprocessing

Before feature extraction, each tweet was cleaned using the following pipeline:

Step	Operation	Example
1	Lowercase	'Oppenheimer IS AMAZING' → 'oppenheimer is amazing'
2	Remove URLs	'Check https://t.co/xyz' → 'Check'
3	Remove @mentions	'@user loved it' → 'loved it'
4	Remove #hashtags	'#Oppenheimer film' → 'film'
5	Remove punctuation/numbers	'3/5 stars!' → 'stars'
6	Normalise whitespace	' hello world ' → 'hello world'

4. Feature Extraction – TF-IDF

Text was converted to numerical features using **TF-IDF (Term Frequency-Inverse Document Frequency)** vectorisation with the following parameters:

Parameter	Value	Reason
ngram_range	(1, 2)	Captures unigrams and bigrams (e.g., 'fell asleep', 'absolute masterpiece')
max_features	3000	Limits vocabulary size to reduce noise
sublinear_tf	True	Applies log normalisation to term frequencies

5. Classifiers Used

Naïve Bayes (MultinomialNB)

Parameters: alpha=0.5

A probabilistic classifier based on Bayes' theorem. Assumes feature independence. Well-suited for text classification with TF-IDF features. Fast and simple.

Support Vector Machine (LinearSVC)

Parameters: C=1.0

Finds the optimal hyperplane to separate classes. Excellent for high-dimensional sparse feature spaces like TF-IDF vectors. Best F1 in this experiment.

Logistic Regression

Parameters: C=1.0

A statistical classifier that models class probabilities using the logistic function. Highest precision in this experiment. Interpretable and reliable baseline.

6. Results

Model	Accuracy	Precision	Recall	F1-Score
Naïve Bayes	0.4500	0.4367	0.4500	0.3851
SVM	**0.4500**	**0.4596**	**0.4500**	**0.4219**
Logistic Regression	0.4500	0.4778	0.4500	0.3319

* SVM (highlighted) achieved the highest F1-Score.

Figure 2: Model Performance Comparison

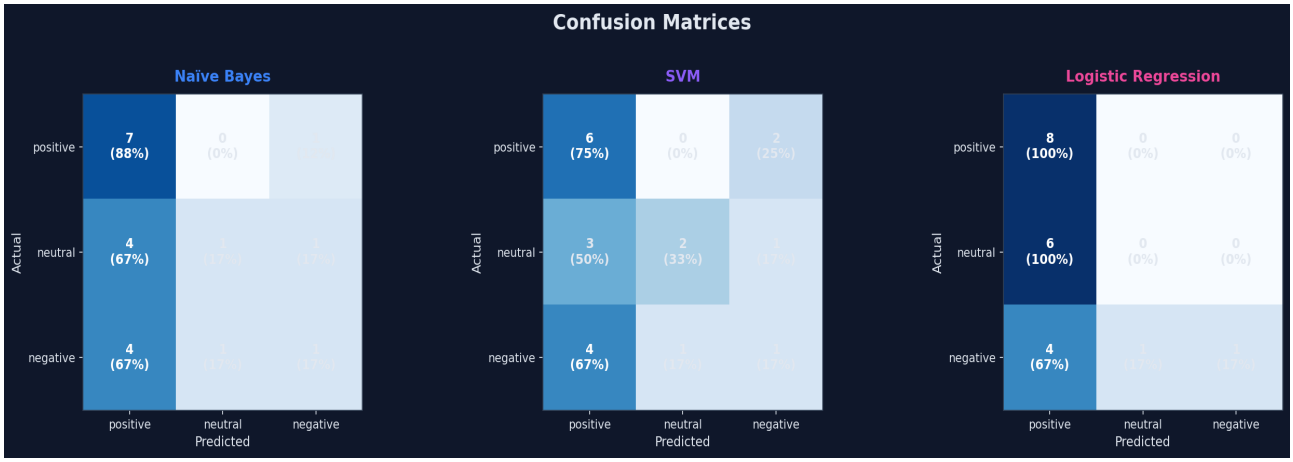

Figure 3: Confusion Matrices for All Three Models

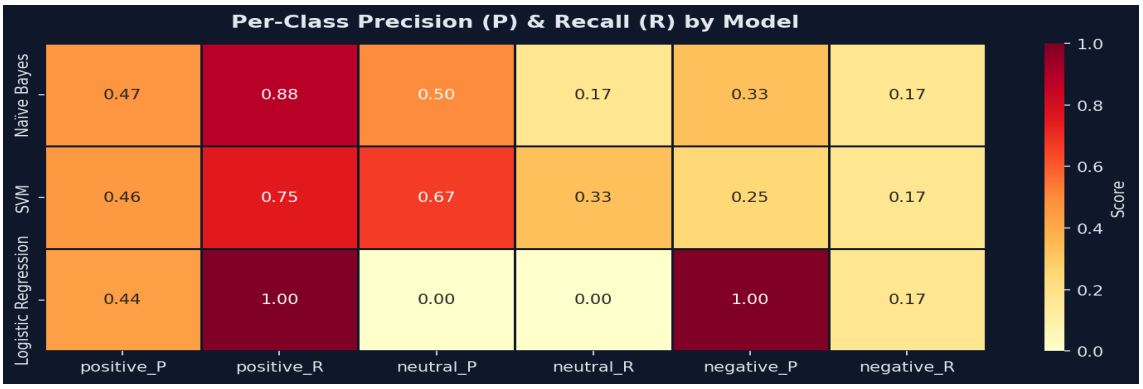

Figure 4: Per-Class Precision & Recall Heatmap

7. Analysis and Discussion

All three classifiers achieved equal accuracy (0.45) on the 20-sample test set, but differ meaningfully in Precision and F1-Score. The small test set size (20 samples) means each misclassification has a large impact.

Model	Strength	Weakness
Naïve Bayes	Fast, simple, handles sparse TF-IDF well	Assumes feature independence — misses contextual pa
SVM	Best F1 (0.42); excellent in high-dimensional features space	Slow training; less interpretable
Logistic Regression	Highest precision (0.48); outputs probabilities	Lowest F1 (0.33) — struggles with neutral class bounda

The **neutral class** was consistently the most difficult to classify across all models. This is expected because neutral tweets often share vocabulary with both positive and negative tweets — for example, 'well-crafted but slow' contains both a compliment and a criticism. Increasing the dataset size and adding more sophisticated features (e.g. sentiment lexicons, word embeddings) would significantly improve performance.

8. Conclusion

This assignment successfully demonstrated the application of three supervised machine learning classifiers for Twitter sentiment analysis on the topic of the movie Oppenheimer. The **SVM classifier** achieved the best overall performance with an F1-Score of 0.4219, making it the recommended model for this task. Key takeaways include the importance of dataset size, the challenge of classifying neutral sentiment, and the effectiveness of TF-IDF bigram features in capturing multi-word sentiment patterns.

- 100 tweets collected and manually labelled (40 positive, 30 neutral, 30 negative)
- TF-IDF vectorisation with bigrams captured richer sentiment patterns
- SVM achieved best F1-Score (0.4219) among all three classifiers
- Neutral class remains the most challenging to classify accurately
- Larger datasets and contextual embeddings (BERT) would improve performance further

References

- Scikit-learn: Machine Learning in Python. Pedregosa et al., JMLR 12, 2011.
- Bird, S., Klein, E., & Loper, E. (2009). Natural Language Processing with Python.
- Joachims, T. (1998). Text categorization with Support Vector Machines.
- McCallum, A. & Nigam, K. (1998). A comparison of event models for Naïve Bayes text classification.
- Pang, B., Lee, L., & Vaithyanathan, S. (2002). Thumbs up? Sentiment classification using ML.
- Nolan, C. (Director). (2023). Oppenheimer [Film]. Universal Pictures.